

DP Computer Science: B 2.2.2 Arrays and Lists (Python)

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this section, students will be able to:

- Understand how **Lists** and **two-dimensional Lists** are applied in Python
 - Understand and code operations that allow **adding, removing, and traversing** elements in dynamic lists
 - Construct programs that effectively utilise **one-dimensional and two-dimensional lists**
-

Key Vocabulary

Term	Definition
List	A dynamic data structure in Python that can hold a collection of elements
Element	An individual item stored in a list
Index	The position of an element in a list (0-indexed)
Traversal	The process of accessing each element in a list
Append	Adding an element to the end of a list
Insert	Adding an element at a specific position in a list
Remove	Deleting an element from a list
2D List	A list of lists; a grid-like structure with rows and columns

ATL Skills

Skill	Description
Thinking	Critical thinking (analysing list operations); application (constructing programs)
Communication	Explaining list operations and code logic
Self-Management	Organising code and problem-solving approaches

2. Introduction to Lists

The Playlist Analogy

| *"Imagine you're creating a playlist for a party. How would you store the list of songs?"*

What is a List?

| A list is a dynamic data structure in Python that can hold a collection of elements. Lists are mutable (can be changed) and can grow or shrink as needed.

Creating Lists

```
python

# Empty list
empty_list = []

# List with elements
songs = ["Bohemian Rhapsody", "Shape of You", "Uptown Funk"]
numbers = [10, 20, 30, 40, 50]
mixed = ["Alice", 25, True, 3.14] # Lists can hold different types

# Using the List() constructor
songs2 = list(["Song 1", "Song 2", "Song 3"])
```

Accessing List Elements

```
python

songs = ["Bohemian Rhapsody", "Shape of You", "Uptown Funk"]

# Access by index (0-indexed)
first = songs[0]    # "Bohemian Rhapsody"
second = songs[1]   # "Shape of You"
last = songs[-1]    # "Uptown Funk" (negative index from the end)

# Accessing elements in a list
print(songs[0])     # Output: Bohemian Rhapsody
print(songs[-1])    # Output: Uptown Funk
```

List Indexing

text

LIST INDEXING					
songs = ["Bohemian Rhapsody", "Shape of You", "Uptown Funk"]					
Positive Index:	0	1	2		
Negative Index:	-3	-2	-1		
<table><tr><td>Bohemian Rhapsody</td><td>Shape of You</td><td>Uptown Funk</td></tr></table>			Bohemian Rhapsody	Shape of You	Uptown Funk
Bohemian Rhapsody	Shape of You	Uptown Funk			
↑	↑	↑			
index 0	index 1	index 2			

3. Basic List Operations

Traversing a List

python

```
songs = ["Bohemian Rhapsody", "Shape of You", "Uptown Funk"]
```

```
# Using a for Loop
```

```
for song in songs:  
    print(song)
```

```
# Using index and range
```

```
for i in range(len(songs)):  
    print(f"Song {i+1}: {songs[i]}")
```

```
# Using enumerate
```

```
for index, song in enumerate(songs):  
    print(f"Index {index}: {song}")
```

Adding Elements

python

```
# append() - adds to the end
```

```
songs = ["Bohemian Rhapsody", "Shape of You"]  
songs.append("Uptown Funk")
```

```
# songs = ["Bohemian Rhapsody", "Shape of You", "Uptown Funk"]
```

```
# insert() - adds at a specific position
```

```
songs.insert(1, "Blinding Lights")
```

```
# songs = ["Bohemian Rhapsody", "Blinding Lights", "Shape of You", "Uptown Funk"]
```

```
# extend() - adds multiple elements
```

```
more_songs = ["Dance Monkey", "Bad Guy"]  
songs.extend(more_songs)
```

```
# songs = ["Bohemian Rhapsody", "Blinding Lights", "Shape of You", "Uptown Funk", "Dance Monkey", "Bad Guy"]
```

Removing Elements

python

```

# remove() - removes the first occurrence of a value
songs = ["Bohemian Rhapsody", "Shape of You", "Uptown Funk", "Shape of You"]
songs.remove("Shape of You") # Removes the first occurrence

# pop() - removes by index (default: Last element)
last = songs.pop()          # Removes and returns the last element
first = songs.pop(0)         # Removes and returns the first element

# del - deletes by index
del songs[1]                 # Deletes the element at index 1

# clear() - removes all elements
songs.clear()                # Removes all elements

```

Other Useful List Methods

Method	Description	Example
<code>len()</code>	Returns the length	<code>len(songs)</code>
<code>index()</code>	Finds the index of a value	<code>songs.index("Shape of You")</code>
<code>count()</code>	Counts occurrences of a value	<code>songs.count("Shape of You")</code>
<code>sort()</code>	Sorts the list in place	<code>songs.sort()</code>
<code>reverse()</code>	Reverses the list in place	<code>songs.reverse()</code>
<code>copy()</code>	Creates a shallow copy	<code>new_list = songs.copy()</code>

4. Common List Patterns

Finding the Largest Number

```
python
```

```
numbers = [45, 78, 23, 89, 12, 56]
```

```
# Method 1: Using max()
```

```
largest = max(numbers)
```

```
print(largest) # Output: 89
```

```
# Method 2: Manual traversal
```

```
largest = numbers[0]
```

```
for num in numbers:
```

```
    if num > largest:
```

```
        largest = num
```

```
print(largest) # Output: 89
```

Finding the Smallest Number

```
python
```

```
numbers = [45, 78, 23, 89, 12, 56]
```

```
# Method 1: Using min()
```

```
smallest = min(numbers)
```

```
# Method 2: Manual traversal
```

```
smallest = numbers[0]
```

```
for num in numbers:
```

```
    if num < smallest:
```

```
        smallest = num
```

Calculating the Average

```
python
```

```
numbers = [45, 78, 23, 89, 12, 56]
```

```
total = sum(numbers)
```

```
average = total / len(numbers)
```

```
print(average) # Output: 50.5
```

```
# Manual traversal
```

```
total = 0
```

```
for num in numbers:
```

```
total += num
average = total / len(numbers)
```

Filtering a List

python

```
numbers = [45, 78, 23, 89, 12, 56, 60, 70]

# Get all numbers greater than 50
filtered = []
for num in numbers:
    if num > 50:
        filtered.append(num)
print(filtered) # Output: [78, 89, 56, 60, 70]

# Using list comprehension
filtered = [num for num in numbers if num > 50]
```

Searching in a List

python

```
names = ["Alice", "Bob", "Charlie", "Diana", "Bob"]

# Check if an item exists
if "Bob" in names:
    print("Bob is in the list")

# Find all occurrences
indices = [i for i, name in enumerate(names) if name == "Bob"]
print(indices) # Output: [1, 4]
```

5. Two-Dimensional (2D) Lists

What is a 2D List?

A 2D list is a list of lists. It represents a grid-like structure with rows and columns.

Creating a 2D List

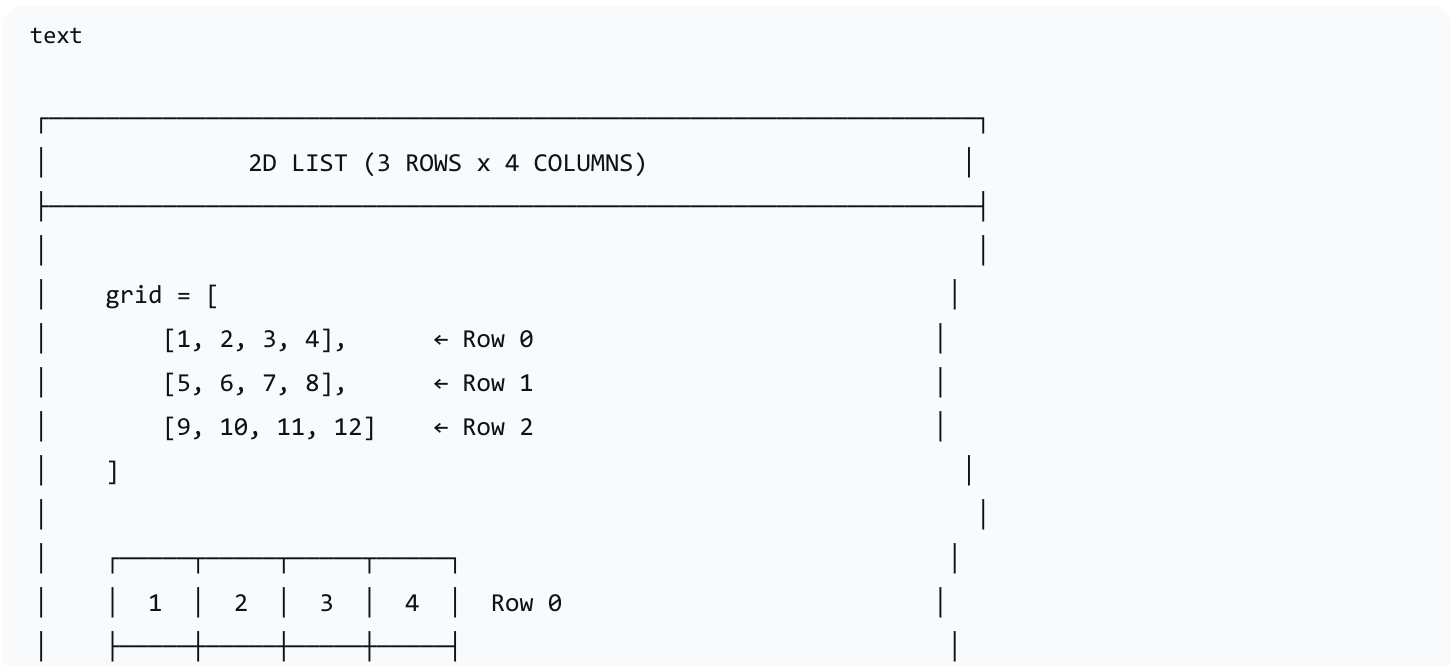
```
python

# A 3x4 grid (3 rows, 4 columns)
grid = [
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12]
]

# Accessing elements
row = grid[0]      # [1, 2, 3, 4]
element = grid[0][2] # 3 (row 0, column 2)

# Creating a 2D List with Loops
rows = 3
cols = 4
grid = [[0] * cols for _ in range(rows)]
# grid = [[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
```

Visual Representation



	5	6	7	8	Row 1
	9	10	11	12	Row 2
	Col 0	Col 1	Col 2	Col 3	
	grid[1][2] = 7 (Row 1, Column 2)				

Traversing a 2D List

python

```
grid = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Using nested loops
for row in grid:
    for element in row:
        print(element, end=" ")
    print()

# Using indices
for i in range(len(grid)):
    for j in range(len(grid[i])):
        print(grid[i][j], end=" ")
    print()
```

Common 2D List Operations

python

```
# Calculate the sum of all elements
grid = [
    [1, 2, 3],
    [4, 5, 6],
```

```

    [7, 8, 9]
]

total = 0
for row in grid:
    total += sum(row)
print(total) # Output: 45

# Find the sum of each row
row_sums = []
for row in grid:
    row_sums.append(sum(row))
print(row_sums) # Output: [6, 15, 24]

# Find the sum of each column
col_sums = [0] * len(grid[0])
for i in range(len(grid)):
    for j in range(len(grid[i])):
        col_sums[j] += grid[i][j]
print(col_sums) # Output: [12, 15, 18]

# Calculate the sum of the main diagonal
diagonal_sum = 0
for i in range(len(grid)):
    diagonal_sum += grid[i][i]
print(diagonal_sum) # Output: 15

```

Applications of 2D Lists

Application	Description
Game Boards	Tic-tac-toe, chess, battleship
Spreadsheets	Grid of data (rows and columns)
Image Processing	Matrix of pixel values
Matrices	Mathematical operations
Maps	Grid-based navigation

6. Worksheet Practice

Part 1: 1D Lists (Basic Understanding)

Task 1: Create and Access

```
python

# Create a List of your 5 favourite movies
# Print the first, third, and last movie
```

Task 2: Add and Remove

```
python

# Start with an empty list
# Add 3 items to the list
# Insert an item at position 1
# Remove the last item
# Print the final list
```

Task 3: Traversal and Analysis

```
python

# Given a list of numbers, find:
# - The largest number
# - The smallest number
# - The average
```

Part 2: 2D Lists

Task 1: Create a 3x3 Grid

```
python

# Create a 3x3 grid of numbers
# Print the grid
```

```
# Access and print the element at (1, 2)
```

Task 2: Calculate Row and Column Sums

```
python
```

```
# Given a 2D List, calculate:  
# - The sum of each row  
# - The sum of each column
```

Task 3: Find the Diagonal Sum

```
python
```

```
# Calculate the sum of the main diagonal  
# (top-left to bottom-right)
```

Challenge Problems

```
python
```

```
# 1. Transpose a matrix  
# 2. Find the maximum element in a 2D List  
# 3. Multiply two matrices  
# 4. Implement a simple game board (e.g., tic-tac-toe)
```

7. Lesson Sequence

Lesson 1: Introduction to Lists and 1D Lists

Phase	Time	Activity
Engage	10 min	Playlist scenario; introduce lists

Phase	Time	Activity
Explore	20 min	Create lists, access elements, basic operations
Explain	15 min	Worksheet (basic understanding)
Elaborate	10 min	Challenge: find largest number

Lesson 2: 2D Lists and List Operations

Phase	Time	Activity
Engage	10 min	Recap; tic-tac-toe board example
Explore	20 min	Create 2D lists; list methods
Explain	15 min	Worksheet (2D lists & operations)
Elaborate	10 min	Group challenge: diagonal sum

Lesson 3: Applications and Problem Solving

Phase	Time	Activity
Engage	10 min	Recap; real-world applications
Explore	20 min	Problem solving (transpose, average, etc.)
Explain	15 min	Walkthrough of challenging problems
Elaborate	10 min	Independent/collaborative practice

8. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Worksheet	Evaluate understanding of list operations
Challenge Problems	Assess problem-solving skills

9. Differentiation

Level	Strategy
Support	Provide visual diagrams and guided examples
Challenge	Sorting algorithms; advanced matrix operations

10. Resources

- Presentation (Slide Deck)
- Worksheet: 1D and 2D list exercises
- Worksheet Answers

11. Summary: Key Takeaways

Concept	Key Point
List	Dynamic collection of elements (mutable)
Index	0-indexed position of elements
Traversal	Accessing each element using loops
Append	Add to the end

Concept	Key Point
Insert	Add at a specific position
Remove/Pop	Remove elements
2D List	List of lists (rows and columns)

text

KEY REMINDERS
✓ Lists are dynamic and mutable ✓ Lists are 0-indexed ✓ append() adds to the end ✓ insert() adds at a specific position ✓ remove() deletes by value ✓ pop() deletes by index ✓ 2D lists = list of lists (grid) ✓ Use nested loops for 2D list traversal

"Lists are one of the most versatile data structures in Python—they allow us to store, access, and manipulate collections of data efficiently."